

MIDI with CircuitPython

- Materials
 - Circuit Playground Bluefruit (CPB)
 - USB cable
- Software
 - [MIDI Monitor](#)
 - Runs in the browser (Chrome, Firefox, Edge) and will let you monitor incoming MIDI data and send MIDI data to the CPB
- Reference Links
 - CircuitPython Library Docs
 - adafruit_midi: <https://docs.circuitpython.org/projects/midi/en/latest/api.html>
 - adafruit_lis3dh: <https://docs.circuitpython.org/projects/lis3dh/en/latest/api.html>
 - neopixel: <https://docs.circuitpython.org/projects/neopixel/en/latest/api.html>
 - simpleio map_range(): <https://docs.circuitpython.org/projects/simpleio/en/latest/examples.html#id3>
 - touchio: <https://docs.circuitpython.org/en/latest/shared-bindings/touchio/>
 - analogio: <https://docs.circuitpython.org/en/latest/shared-bindings/analogio/index.html>
 - adafruit_ble_midi: https://docs.circuitpython.org/projects/ble_midi/en/latest/api.html
 - adafruit_debouncer: <https://docs.circuitpython.org/projects/debouncer/en/latest/api.html>
 - MIDI Links
 - [Control Change Message Table](#)
 - [MIDI Note Numbers](#)

NeoPixel MIDI Listener

- USB MIDI init:
 - `midi = adafruit_midi.MIDI(midi_in=usb_midi.ports[0], in_channel=0)`
- NeoPixel init:
 - `pixels = neopixel.NeoPixel(board.NEOPIXEL, 10, brightness=0.2, auto_write=True)`
- Create two colors using RGB values
 - Ex: RED = (255, 0, 0)
- In the loop, listen for MIDI messages with:
 - `msg = midi.receive()`
- Check message type with:
 - if msg is not None:
 - `if isinstance(msg, NoteOn):`
- [Solution on GitHub](#)

Touch MIDI Keyboard

- USB MIDI init:
 - `midi = adafruit_midi.MIDI(midi_out=usb_midi.ports[1], out_channel=0)`
- Lists
 - Lists are a great way to coordinate matched sets of data because you access the entries by index. Therefore, you can have a MIDI note at index 2 in a list match a touch input at index 2 in another list
 - Create a list of MIDI note numbers, the same number of touch inputs (seven):
 - `midi_notes = [60, 62, 64, etc]`
 - Create a list of the touch pins:
 - `pins = [board.A1, board.A2, board.A3, board.A4, board.A5, board.A6, board.TX]`
 - Create two empty lists:
 - `touch_inputs = []`
 - `touch_states = []`
 - These will hold the touchio inputs and the state of the touch input for debouncing
- Use a for statement to iterate through the pins list to set them as TouchIn inputs and set their current touch states as False
 - for touch in pins:
 - `touch_pin = touchio.TouchIn(touch)`
 - `touch_inputs.append(touch_pin)`
 - `state = False`
 - `touch_states.append(state)`
- In the loop, if touched, send a MIDI NoteOn message for the corresponding MIDI note, else send a MIDI NoteOff message for the same note
 - Use a for statement to iterate:
 - for i in range(len(touch_inputs)):
 - `inputs = touch_inputs[i]`
 - Check state with:
 - if `inputs.value` and `touch_states[i]` is False:
 - if not `inputs.value` and `touch_states[i]` is True:
 - Set state with:
 - `touch_states[i] = True`
 - `touch_states[i] = False`
 - Set the state to True if the touch input is True, set to False if not touched to reset
- [Solution on GitHub](#)

Change Octaves

- Build on top of the Touch MIDI Keyboard
- The onboard buttons will change the octave that the touch inputs are sending. Begin with a starter list of notes and set the octave range:
 - `BASE_NOTES = [60, 62, 64, 65, 67, 69, 71]` # C scale from middle C
 - `midi_notes = BASE_NOTES`
 - `MIN_OCTAVE = -4` # C0 (MIDI 12)
 - `MAX_OCTAVE = 3` # C7 (MIDI 96)
 - `octave = 0`
- Initialize the onboard buttons with Debouncer:
 - `buttona = digitalio.DigitalInOut(board.BUTTON_A)`
 - `buttona.switch_to_input(pull=digitalio.Pull.DOWN)`
 - `octave_down = Debouncer(buttona)`

 - `buttonb = digitalio.DigitalInOut(board.BUTTON_B)`
 - `buttonb.switch_to_input(pull=digitalio.Pull.DOWN)`
 - `octave_up = Debouncer(buttonb)`
- How to shift the notes?
 - Use a function that takes the starter list of notes and adds the octave range:
 - `def shifted_notes():`
 - `return [n + octave * 12 for n in BASE_NOTES]`
 - What is that weird line of code doing? It's an abbreviated expression that is:
 - `list = []`
 - `for n in BASE_NOTES:`
 - `note = n + octave * 12`
 - `list.append(note)`
 - `midi_notes = array`

Change Octaves, cont'd

- In the loop, make sure to update the buttons:
 - `octave_down.update()`
 - `octave_up.update()`
- If the buttons are pressed and the octave range has not gone past the `MIN_` and `MAX_` values set at the top of the code, update the `midi_notes` list with the `shifted_notes()` function
 - if `octave_down.fell` and `octave > MIN_OCTAVE`:
 - `octave -= 1`
 - `midi_notes = shifted_notes()`
 - `print(octave, midi_notes)`
 - if `octave_up.fell` and `octave < MAX_OCTAVE`:
 - `octave += 1`
 - `midi_notes = shifted_notes()`
 - `print(octave, midi_notes)`
- [Solution on GitHub](#)

Light Sensor to CC Message

- The onboard light sensor reads out as an analog value (range of 0 to 65535)
- Use the same MIDI init:
 - `midi = adafruit_midi.MIDI(midi_out=usb_midi.ports[1], out_channel=0)`
- Init the light sensor as an analog input:
 - `light = AnalogIn(board.LIGHT)`
- You'll compare the current light sensor value to the previous light sensor value. Create a variable to hold the previous value:
 - `previous_modulation = 0`
- In the loop, use `map_range` to map the light sensor value range to the CC message value range:
 - `mod_val = simpleio.map_range(light.value, 0, 65535, 0, 127)`
 - This returns a value that can be sent as a CC message
- If the value difference is greater than 2, then send a new CC value:
 - `if abs(mod_val - previous_modulation) > 2:`
 - `# update mod_val2`
 - `previous_modulation = mod_val`
 - `# create integer`
 - `modulation = int(mod_val)`
 - `# create CC message`
 - `modWheel = ControlChange(1, modulation)`
 - `# send CC message`
 - `midi.send(modWheel)`
- All CC messages need to be sent as an integer, `map_range()` returns a float
- [Solution on GitHub](#)

Accelerometer to PitchBend

- Same USB MIDI init:
 - `midi = adafruit_midi.MIDI(midi_out=usb_midi.ports[1], out_channel=0)`
- LIS3DH I2C init:
 - `i2c = busio.I2C(board.ACCELEROMETER_SCL, board.ACCELEROMETER_SDA)`
 - `lis3dh = adafruit_lis3dh.LIS3DH_I2C(i2c, address=0x19)`
 - `lis3dh.range = adafruit_lis3dh.RANGE_2_G`
- Similar to sending CC, you'll track the previous value to compare against:
 - `no_bend = 8192` # center pitch bend value for no bend
 - `last_bend = 8192`
 - The center value/no bend for PitchBend is 8192
- You'll make a "debounce" value to quiet the center value from the accelerometer:
 - `debounce = 0.1`
- In the loop, you'll read the values from the accelerometer:
 - `x, y, z = (value / adafruit_lis3dh.STANDARD_GRAVITY for value in lis3dh.acceleration)`
 - The y axis will be used for PitchBend
- In the CPB is flat on a surface, no PitchBend is sent:
 - `if abs(y) < debounce:` # if basically level on y axis, no pitch bend
 - `bend = no_bend`
 - `else:`
 - `bend = no_bend + int(y * (8191 if y > 0 else 8192))`

Accelerometer to PitchBend cont'd

- Clamp the bend value to the PitchBend range:
 - `bend = min(16383, max(0, bend))` # clamp values to pitchbend range
- If the value has changed enough or PitchBend is off, send the value:
 - if `abs(bend - last_bend) > 75` or `(bend == no_bend and last_bend != no_bend)`:
 - `last_bend = bend`
 - `midi.send(PitchBend(bend))`
- [Solution on GitHub](#)
- You can experiment with using a different axis or figuring out tilt to tie to PitchBend or any other CC value

BLE MIDI

- We won't try it in class because it's a little more finicky
 - Might have connection issues with the number of devices trying to start up
 - Might be fun for final projects though
- The only difference is the MIDI init:
 - `# BLE MIDI setup`
`midi_service = adafruit_ble_midi.MIDIService()`
 - `advertisement = ProvideServicesAdvertisement(midi_service)`
 - `ble = adafruit_ble.BLERadio()`
 - `if ble.connected:`
 - `for c in ble.connections:`
 - `c.disconnect()` # midi setup
 - `midi = adafruit_midi.MIDI(midi_out=midi_service, out_channel=0)`
 - `print("advertising")`
 - `ble.start_advertising(advertisement)`
- In the loop:
 - `while True: # BLE connection`
 - `print("Waiting for connection")`
 - `while not ble.connected:`
 - `pass`
 - `print("Connected")`
 - `time.sleep(1.0)`
 - `while ble.connected:`
 - `midi.send(NoteOn(64, 120))`
- [Example on GitHub](#)